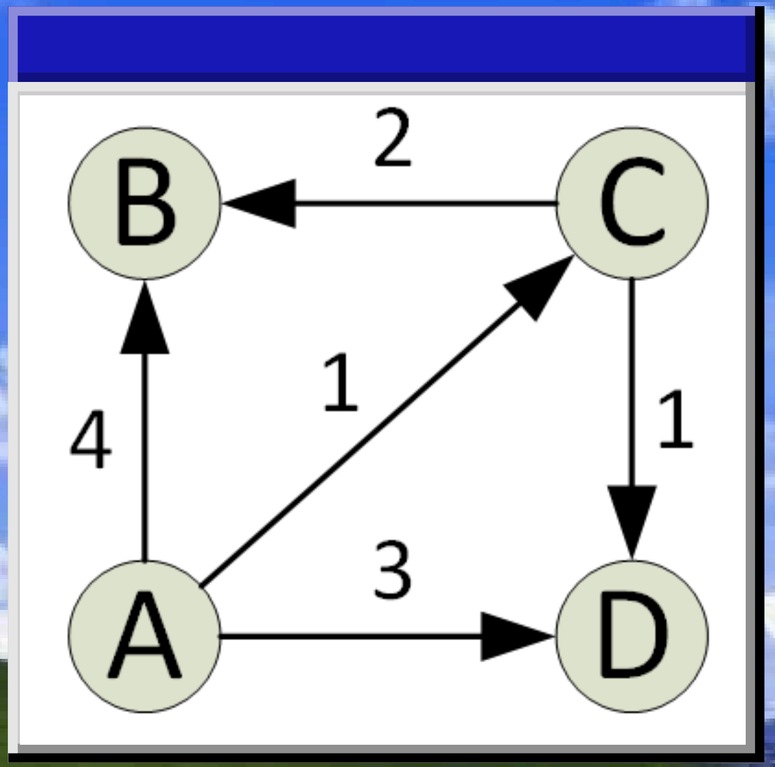
First algorithm

Dijkstra

visual representation



O problema, e onde se usa:

O algoritmo de Dijkstra é um algoritmo para encontrar os caminhos mais curtos entre nós em um grafo ponderado.

Dijkstra pode ser usado para encontrar a rota mais curta entre uma cidade e todas as outras cidades, Jogos, roteamento de redes, etc. Um exemplo é o protocolo de roteamento de rede OSPF (Open Shortest Path First).

11:11PM

Algorithm

THE IDEA

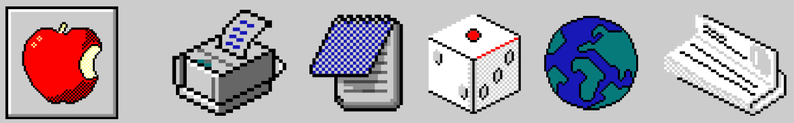
Distância inicial: Infinita para todos e 0 para a origem.

Repetir: escolher o vértice não visitado de menor distância, marcá-lo como visitado e relaxar os vizinhos.

Relaxar: atualizar a distância de um vizinho se passar pelo vértice escolhido for mais curto.

OBS:

com pesos positivos, o vértice de menor distância já tem o valor definitivo. Por isso o código recusa peso menor ou igual a zero.

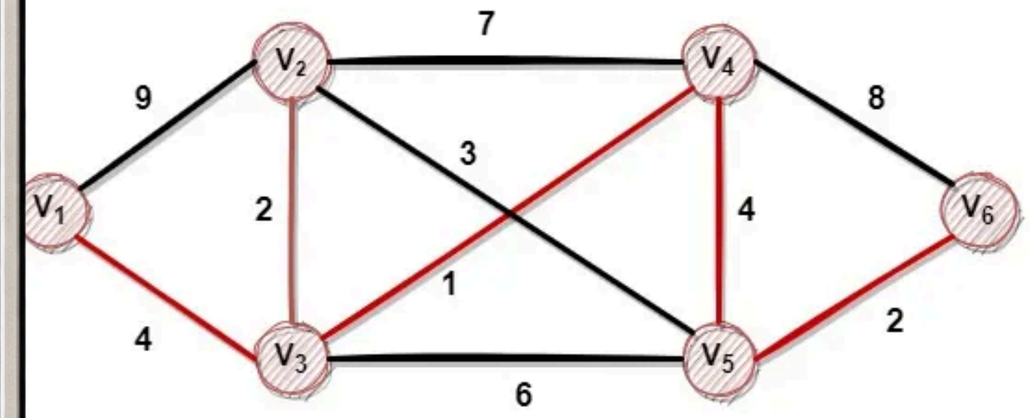


11:11PM

Example:

Untitled

Dijkstra's Algorithm



	Distance	Previous Node	Visited
V_1	0	None	True
V_2	6	V_3	True
V_3	4	V_1	True
V_4	5	V_3	True
V_5	9	V_4	True
V_6	11	V_5	True

Path: V_1 - V_3 - V_4 - V_5 - V_6

Total Cost: 11

Back to Agenda Page

Limits and:

Complexity

Complexidade: O(V^2) busca linear do menor;

Não funciona com pesos negativos;

A matriz gasta memória V×V; lista de adjacência é melhor para grafos grandes.

Vértice sem caminho fica com distância infinita.;

Back to Agenda Page

```c
void dijkstra(Grafo *g, int origem)
{
    int i, u, v;
    int distancia[MAX];
    int visitado[MAX];
    int nova_distancia;
    if((g == NULL) || (origem < 0) || (origem >= 4))
    {
        return;
    }
    inicializar_distancia(distancia, origem);
    inicializar_visitado(visitado);
    for(i = 0 ; i < MAX ; i++)
    {
        u = encontrar_menor(distancia, visitado);
        if(u == -1)
        {
            break;
        }
        visitado[u] = 1;
        for(v = 0 ; v < MAX ; v++)
        {
            if(g->matriz[u][v] != 0 && visitado[v] != 1)
            {
                nova_distancia = distancia[u] + g->matriz[u][v];
                if(nova_distancia < distancia[v])
                {
                    distancia[v] = nova_distancia;
                }
            }
        }
    }
    imprimir_distancia(distancia);
```

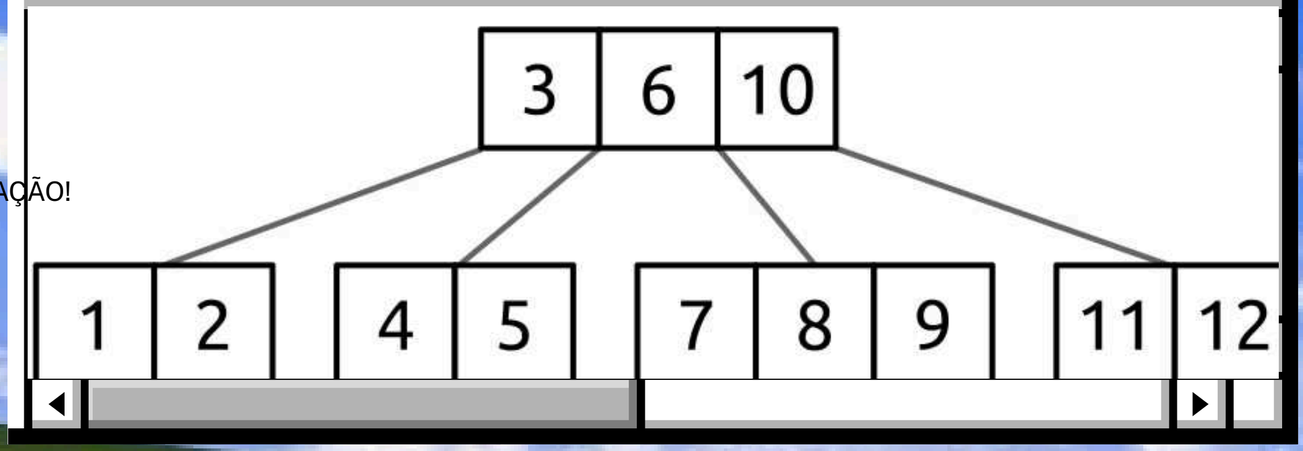


Next algorithm

Árvore B

visual representation

low and wide



AÇÃO!

Sobre:

Uma árvore B é uma estrutura de dados em árvore auto-balanceada que mantém dados ordenados e permite buscas, acesso sequencial, inserções e remoções em tempo logarítmico.

A árvore B generaliza a árvore de busca binária , permitindo que os nós tenham mais de dois filhos. dessa forma a árvore B reduz a altura da árvore e coloca os dados em menos blocos separados.

11:11PM

definition and:

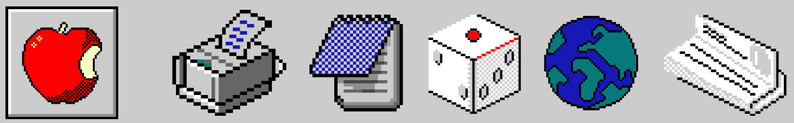
Rules

De acordo com a definição de Knuth , uma árvore B de ordem m é uma árvore que satisfaz as seguintes propriedades:

1. Cada nó tem no máximo m filhos.
2. Cada nó, exceto a raiz e as folhas, tem pelo menos ⌈ m /2⌉ filhos.
3. O nó raiz tem pelo menos dois filhos, a menos que seja uma folha.
4. Todas as folhas aparecem no mesmo nível.
5. Um nó não-folha com k filhos contém k − 1 chaves

OBS:

a regra 2 exclui as folhas, então ela não diz quantas chaves uma folha deve ter. No código, as folhas (menos a raiz) também têm pelo menos t-1 chaves. Também no código, t é o grau mínimo, e a ordem é m = 2t



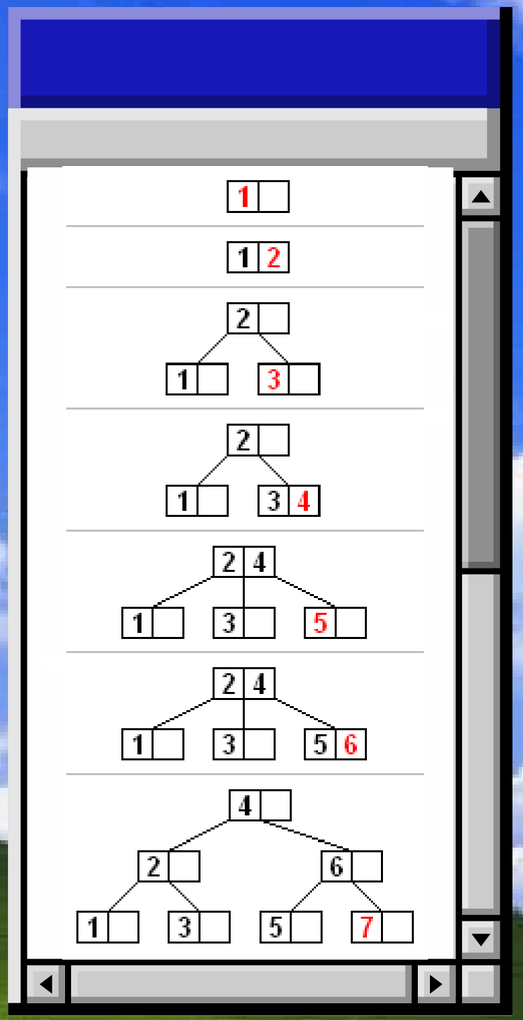
11:11PM

Now:

Insertion

Todas as inserções começam em um nó folha. Para inserir um novo elemento, procure na árvore o nó folha onde o novo elemento deve ser adicionado. Insira o novo elemento nesse nó seguindo os passos abaixo:

1. **Se o nó contiver menos elementos do que o número máximo permitido, haverá espaço para o novo elemento. Insira o novo elemento no nó, mantendo a ordem dos elementos existentes.**
2. **Caso contrário, o nó estará cheio; divida-o igualmente em dois nós, da seguinte forma:**
   a. **Uma única mediana é escolhida dentre os elementos da folha e o novo elemento que está sendo inserido.**
   b. **Valores menores que a mediana são colocados no novo nó esquerdo, e valores maiores que a mediana são colocados no novo nó direito, com a mediana atuando como um valor de separação.**
   c. **O valor de separação é inserido no nó pai, o que pode causar sua divisão, e assim por diante. Se o nó não tiver pai (ou seja, se o nó for a raiz), crie uma nova raiz acima deste nó (aumentando a altura da árvore).**



Back to Agenda Page

Now:

Elimination

Eliminação

Existem duas estratégias populares para a remoção de elementos de uma árvore B.

1. **Localize e exclua o item e, em seguida, reestruture a árvore para manter suas invariantes, OU**
2. **Faça uma única passagem pela árvore, mas antes de entrar (visitar) um nó, reestruture a árvore de forma que, assim que a chave a ser excluída for encontrada, ela possa ser excluída sem a necessidade de qualquer reestruturação adicional.**

Existem dois casos especiais a serem considerados ao excluir um elemento:

1. **O elemento em um nó interno é um separador para seus nós filhos.**
2. **A exclusão de um elemento pode fazer com que seu nó fique com o número mínimo de elementos e filhos.**

Os procedimentos para esses casos estão listados abaixo.

Remoção de um nó folha

1. **Procure o valor que deseja excluir.**
2. **Se o valor estiver em um nó folha, basta excluí-lo do nó.**

rebalancing

O algoritmo para rebalancear a árvore é o seguinte:

Se o nó deficiente tiver um nó irmão à direita com mais elementos do que o número mínimo exigido, então rotacione para a esquerda.

Copie o separador do nó pai para o final do nó deficiente (o separador se move para baixo; o nó deficiente agora tem o número mínimo de elementos).

Substitua o separador no elemento pai pelo primeiro elemento do elemento irmão direito (o elemento irmão direito perde um nó, mas ainda mantém pelo menos o número mínimo de elementos).

A árvore agora está equilibrada.

Caso contrário, se o nó irmão à esquerda do nó deficiente existir e tiver mais elementos do que o número mínimo exigido, então rotacione para a direita.

Copie o separador do nó pai para o início do nó deficiente (o separador se move para baixo; o nó deficiente agora tem o número mínimo de elementos).

Substitua o separador no elemento pai pelo último elemento do elemento irmão esquerdo (o elemento irmão esquerdo perde um nó, mas ainda mantém pelo menos o número mínimo de elementos).

A árvore agora está equilibrada.

Caso contrário, se ambos os irmãos imediatos tiverem apenas o número mínimo de elementos, então a junção será feita com um irmão que contenha o separador removido do elemento pai.

Copie o separador para o final do nó esquerdo (o nó esquerdo pode ser o nó deficiente ou o nó irmão com o menor número de elementos).

Mova todos os elementos do nó da direita para o nó da esquerda (o nó da esquerda agora tem o número máximo de elementos e o nó da direita está vazio).

Remova o separador do elemento pai juntamente com seu filho direito vazio (o elemento pai perde um elemento).

Se o nó pai for a raiz e agora não tiver elementos, libere-o e faça do nó mesclado a nova raiz (a árvore fica mais rasa).

Caso contrário, se o pai tiver menos elementos do que o número necessário, então reequilibre o pai [23]

Back to Agenda Page

Limits and:

Complexity

Para procurar, inserir e excluir, a complexidade é O(log n) para o pior caso e para o caso amortizado.

a análise amortizada é um método para analisar a complexidade de um determinado algoritmo , ou quanto recurso, especialmente tempo ou memória, é necessário para executá-lo . A motivação para a análise amortizada é que observar o tempo de execução no pior caso pode ser muito pessimista. Em vez disso, a análise amortizada calcula a média dos tempos de execução das operações em uma sequência ao longo dessa sequência. Como conclusão: "A análise amortizada é uma ferramenta útil que complementa outras técnicas, como a análise do pior caso e a análise do caso médio ."



Back to Agenda Page